

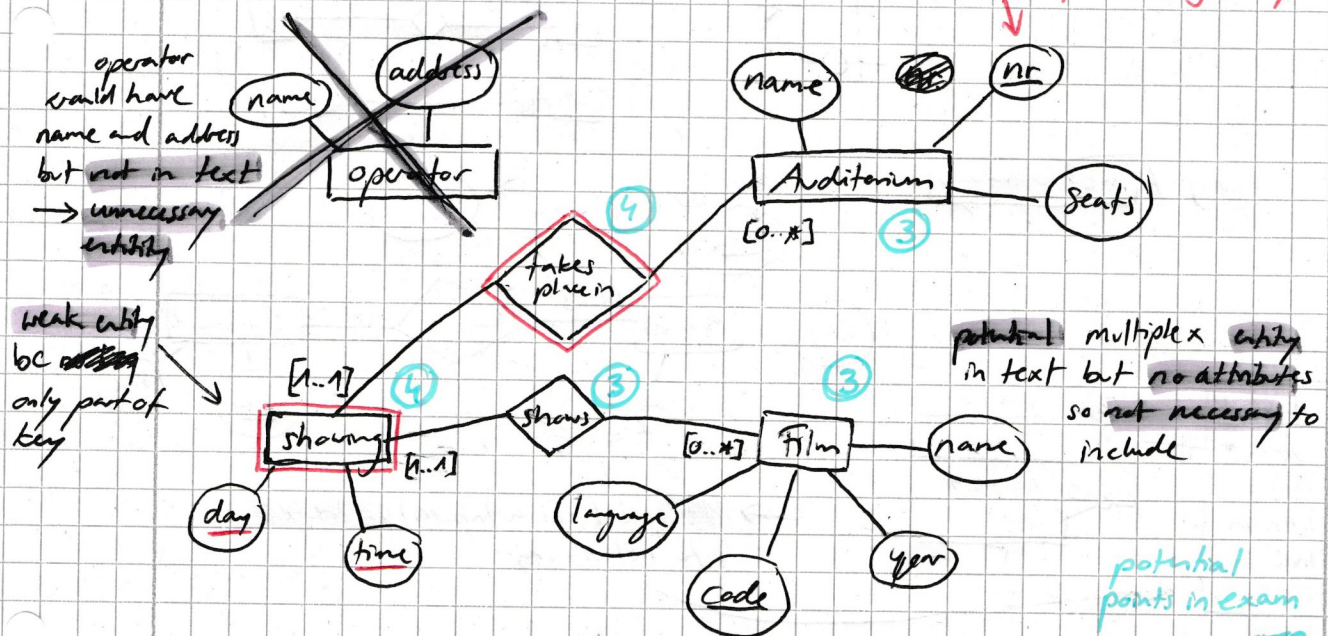
DB

25/11/25

Modelling Guideline ER

- simple and precise with meaningful names that reflect reality
 - entities with no attributes ^{might be} → unnecessary?
 - rel. cycles can be hint of redundant modelling
- ① model entity with complete set of attributes ~ 2P
 - ② relationships and quantization ~ 3P
 - ③ underline keys ~ 1P
 - ④ mark weak entities if no key or just part of key and supporting relationships ~ 4P

Ex: Multiplex Cinema



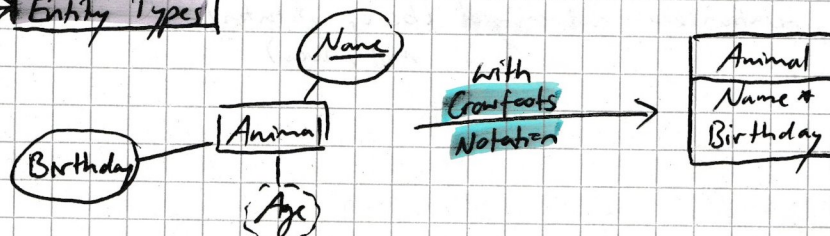
key

showing:	<u>day</u>	<u>time</u>	film code	<u>Auditorium nr</u>
	Fr	20:00	7	A

ER Mapping

- mapping of ER model to relational model → every entity becomes relation
- every relationship becomes relation
- ↳ but: merging of relations, weak entities is - a relationships need to be resolved

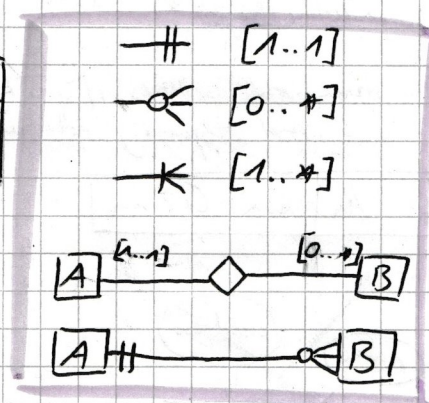
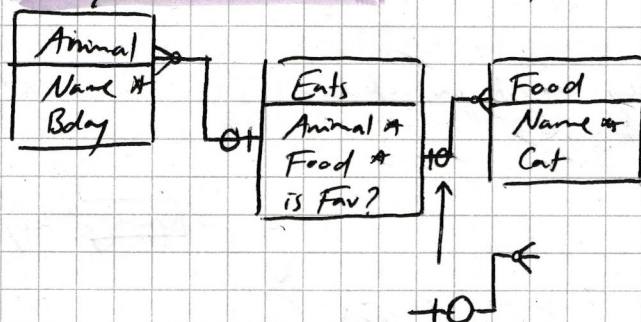
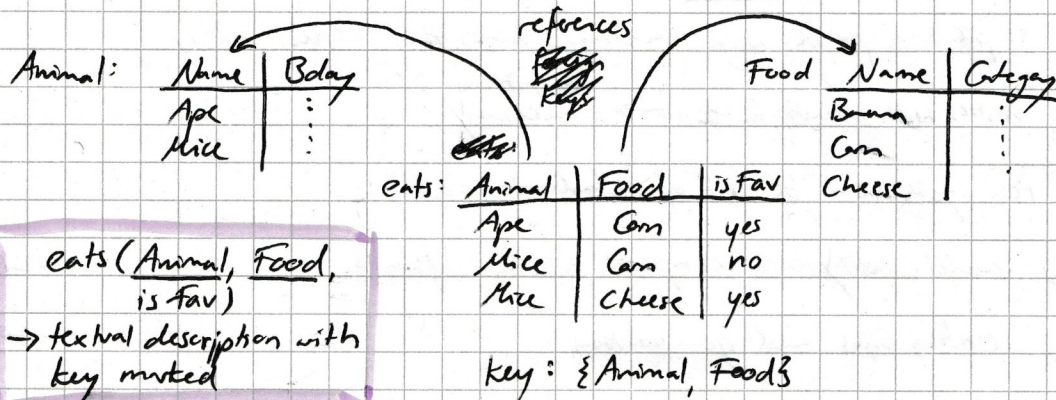
Entity Types



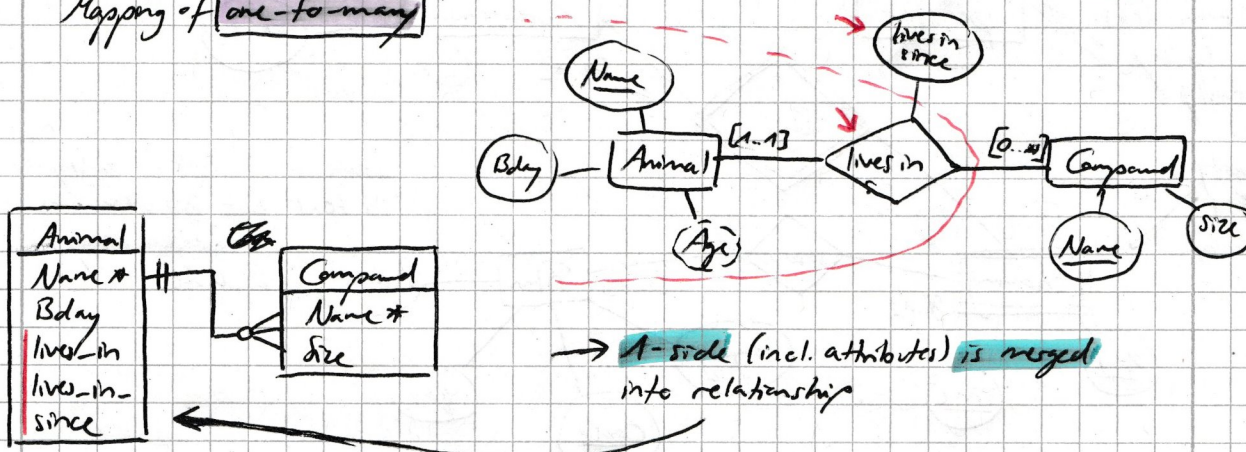
DB

contd.
25/11/25

Mapping of many-to-many relationships



Mapping of one-to-many

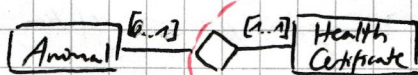


→ in 1-1 case, both entities and relationships can be merged

→ in 0-1 case, the relationship is merged to the 1 side

ex: Health Certificate

→ could be put of on relation with Animal but better to put it into its own to save disk space in case of null values



AHC:

Animal/Name	...	HC
A		X
B		null

→ better

HC	No	Animal

no null values

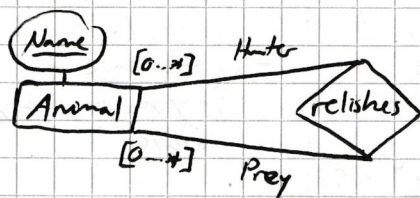
→ in 0-0 case, relationship is not merged (bc of potential null values)

DBS

contd.

25/11/25

Mapping of many relationships (1)

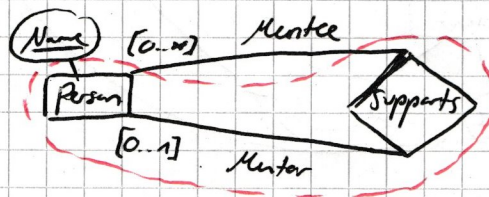


point both to Animal

relishes:

Hunter	Prey
Ape	Kick
Kick	Warm
Ape	Frog
Frog	Fly
Frog	Warm

Mapping of many relationships (2)



Person	Mentor
A	null
B	A
C	A
D	A
E	D

- mentee is stored implicitly

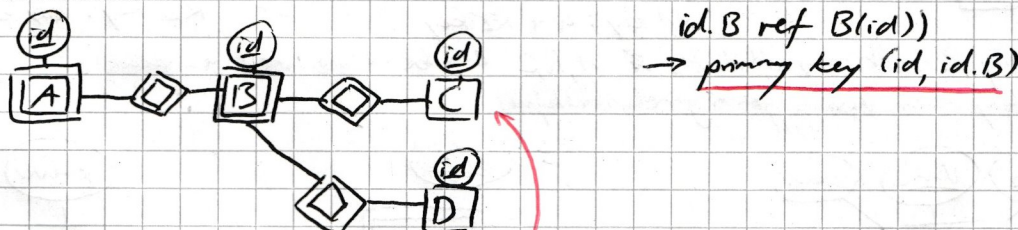
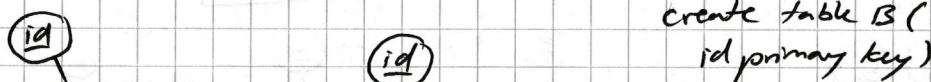
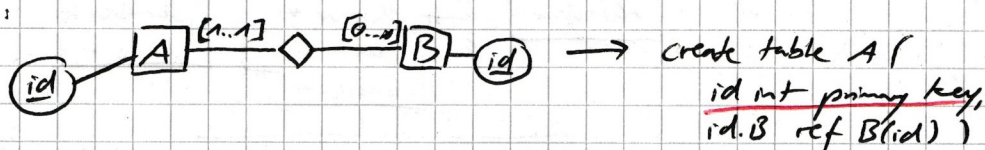
→ merged to 1-side, Null, not null for quantization

Mapping of weak entity

- weak entities ^{"collect"} merge all key attributes from all supporting relationships → become primary key

- supporting relationships always merge to the 1-side

ex.:



create A

id
id.B
id.C
id.D

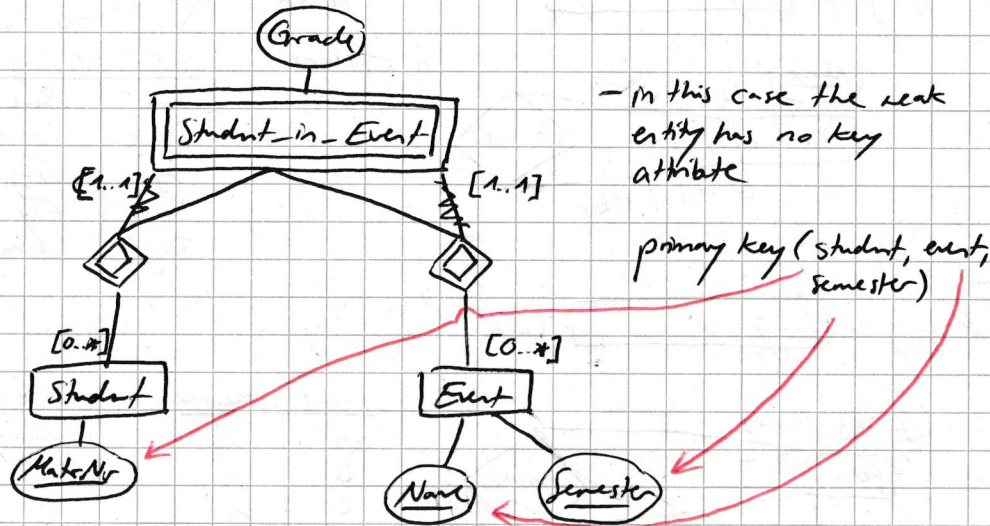
create B

id
id.C
id.D

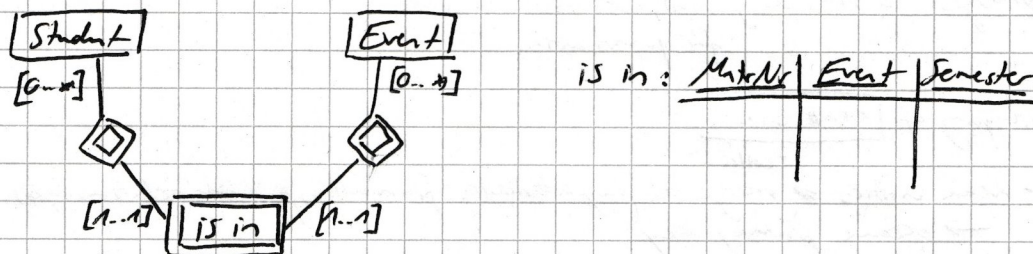
foreign key (id.B, id.C, id.D) references B(id, id.C, id.D)

DB1 contd.
25/11/25

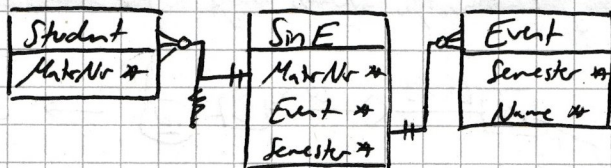
further ex: Student-in-Event



alternate interpretation that result in diff. diagram but same table/key:

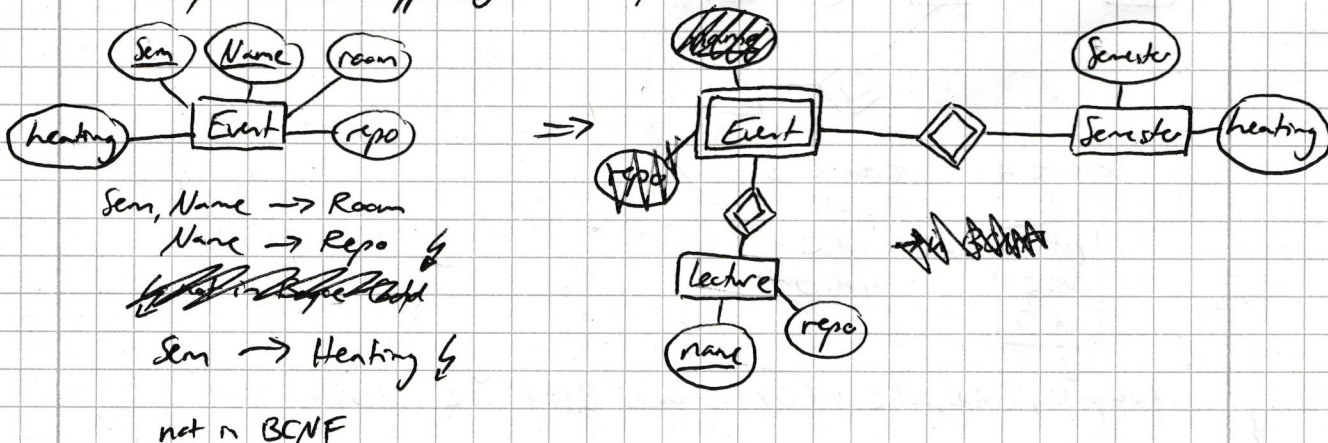
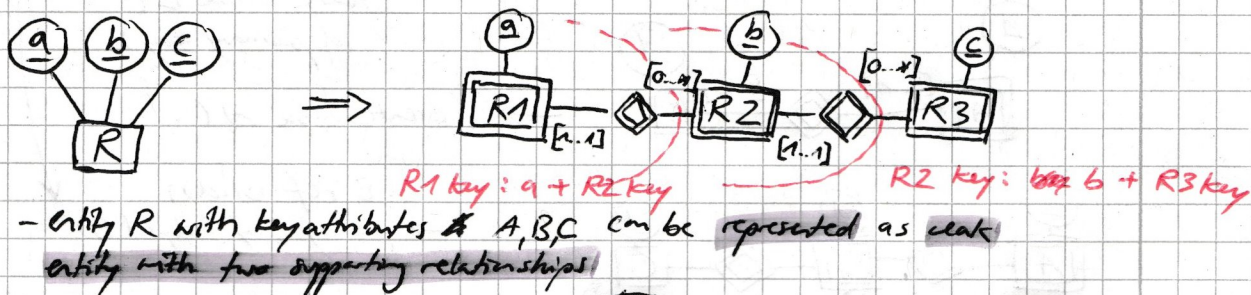


Crowfoot Notation:



DB1
contd.
26/11/25

Multiple key att can always be distributed via weak identities



D13

card.

26/11/25

$$R_1 = \{ \text{name} \}^+ = \{ \text{repo}, \text{name} \}$$

$$R_2 = \{ \text{sem}, \text{name}, \text{heating} \} \not\subseteq \text{BCNF}$$

room

Decomposition

$$R_{2.1} = \{ \text{sem} \}^+ = \{ \text{sem}, \text{heating} \} \quad \text{sem} \rightarrow \text{heating}$$

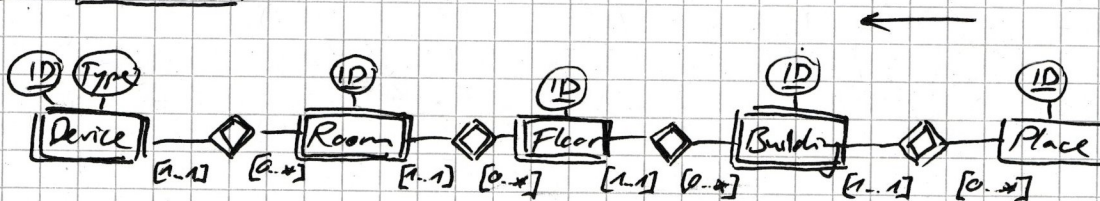
$$R_{2.2} = \{ \text{sem}, \text{name}, \text{room} \} \quad \text{sem}, \text{name} \rightarrow \text{room}$$

→ BCNF

Ex: Derivation of relation

- When resolving an ER diagram, ~~it had to be~~ its rank ~~relationships~~ entities need to be solved in order

Ex: Device ID



Mapping of is-a relationships

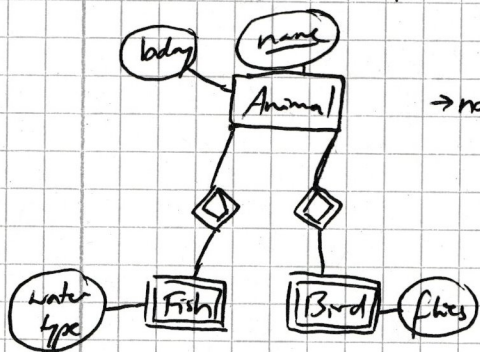
- null value style: put everything into one relation, null values where attribute was not applicable for entry, simple interaction but null values

- ER style: ~~more relations~~ one relation per entity, more relations and no nulls but interaction address more tables

~~OO style~~

- OO style: each subset of all entities becomes a relation, key hard to ~~maintain~~ maintain but only one table addressed for inserts, many relations
object oriented

→ depending on use case null value can still be viable, OO has rather specific the cases due to its complexity



→ now automatically in ER style

→ fewer relations generally better since it performs better (needs less joins in interaction)

- less space required is generally better